Vehicle Service Booking System

Thilini Lakpriya Hettiarachchi

Supervisor-Miss Imasha Ariyasinghe

Contents

Introduction and Problem Statement.....3

System Requirement Specification (SRS) 4

System Design & Architecture.....6

Database Design 8

User Interface (UI) Design 12

Security, Testing, and Deployment.....14

Conclusion and Future Enhancements 15

1. Introduction and Problem Statement

1.1 Introduction

The automotive service industry is currently undergoing a massive digital transformation. In an era where everything from groceries to banking is managed via smartphones, many vehicle service centers still rely on traditional, manual methods to handle their day-to-day operations. This project, the **Vehicle Service Booking System**, was developed to modernize these operations using the power of the **MERN Stack** (MongoDB, Express, React, and Node.js).

The primary goal of this application is to provide a seamless digital bridge between the vehicle service provider and the customer. For the customer, it offers a hassle-free way to book appointments and track their vehicle's progress in real-time. For the administrator, it provides a centralized dashboard to manage schedules, track performance, and organize the workflow efficiently. This project wasn't just about building a website; it was about creating a solution that saves time, reduces human error, and improves the overall customer experience in a competitive service market.

1.2 Problem Statement

During my initial research and observation, I identified several critical "pain points" in the manual management systems used by local service centers. These issues not only affect the staff but also create a negative impression on the customers.

- **Booking Conflicts & Overlaps:** Without a centralized digital calendar, staff often double-book service slots, leading to long waiting times for customers and unnecessary pressure on the mechanics.
- **Lack of Real-time Updates:** Currently, a customer has to call the service center repeatedly or physically visit to know if their vehicle is ready. This lack of transparency leads to frustration.
- **Data Vulnerability:** Relying on physical logbooks or spreadsheets makes it incredibly difficult to retrieve historical data. If a book is misplaced or damaged, years of customer records and vehicle history could be lost forever.
- **Inefficient Management:** For the owner or manager, getting a clear picture of how many bookings are pending or completed in a day is a time-consuming task that requires manual counting and cross-checking.

1.3 Project Objectives

To address the problems mentioned above, I defined a set of clear objectives for this project:

- **To automate the appointment process:** Create a 24/7 accessible online platform where customers can submit booking requests without needing to make a phone call.
- **To provide real-time status visibility:** Implement a tracking module where customers can enter their vehicle number to see exactly which stage their service is in (e.g., Pending, Working, or Completed).

- **To centralize administrative control:** Build a secure dashboard for admins to oversee all bookings, update statuses with a single click, and view daily service metrics.
- **To ensure data integrity and security:** Utilize a cloud-based NoSQL database (MongoDB) to store records safely and implement modern security protocols (JWT, Sanitization) to protect sensitive information.

1.4 Scope of the Project

The scope of this system is focused on the core operations of a service center during this initial development phase:

- **In-Scope:** Customer appointment scheduling, unique vehicle tracking, admin authentication, booking management (approval/completion), and a responsive user interface for mobile and desktop.
- **Out-of-Scope:** For this 1-month development cycle, features such as automated SMS/Email alerts, online payment gateway integration, and inventory/parts management were kept out of the basic scope but are considered for future updates.

2. System Requirement Specification (SRS)

2.1 Introduction to Requirements

Before writing a single line of code, it was essential to understand what exactly the system needed to do. The requirements gathering phase involved analyzing the needs of two primary user groups: the **Customer** (who needs a quick way to book and track services) and the **Administrator** (who needs to manage the workflow). This chapter outlines the functional and non-functional requirements that formed the blueprint of the Vehicle Service Booking System.

2.2 Functional Requirements

Functional requirements define the core actions the system must perform. I categorized these into two main modules:

2.2.1 Customer Module

- **FR 1.1 - Appointment Booking:** The system must allow users to submit a booking form containing their name, contact details, vehicle number, preferred date, and service type.
- **FR 1.2 - Real-time Status Check:** A public interface where users can input their vehicle number and instantly see if their service is 'Pending', 'Approved', or 'Completed'.
- **FR 1.3 - Input Validation:** The system should ensure that all mandatory fields are filled and that the vehicle number follows a valid format before submission.

2.2.2 Administrative Module

- **FR 2.1 - Secure Login:** Only authorized personnel should be able to access the dashboard through a secure, password-protected login page.

- **FR 2.2 - Booking Management:** Admins must have the ability to view all incoming requests and update their status (e.g., from 'Pending' to 'Approved').
- **FR 2.3 - Record Deletion:** The ability to remove old or canceled records from the system to keep the database organized.
- **FR 2.4 - Live Dashboard Overview:** A quick summary showing the total number of bookings and their current distribution by status.

2.3 Non-Functional Requirements (NFR)

While functional requirements define *what* the system does, non-functional requirements define *how* it performs. These were critical to ensuring a professional user experience.

- **NFR 1 - Performance:** The system should be lightweight. Using React on the frontend and a NoSQL database ensures that page loads and data retrieval happen within a few seconds.
- **NFR 2 - Security:** Sensitive data, such as database credentials and user passwords, must be protected using Environment Variables (.env) and encryption (Bcrypt/JWT).
- **NFR 3 - Scalability:** By utilizing MongoDB Atlas, the system is designed to handle an increasing number of booking records without any loss in performance.
- **NFR 4 - Responsiveness:** The UI must be fully responsive. Whether the admin is using a desktop or the customer is using a mobile phone, the layout must adapt perfectly.
- **NFR 5 - Usability:** The interface should be intuitive, requiring zero training for a new customer to successfully book a service.

2.4 System Environment (Tech Stack)

To meet the above requirements, I selected a modern tech stack that offers flexibility and speed:

- **Frontend:** React.js for building a dynamic, single-page user interface.
- **Backend:** Node.js with Express.js for handling the server-side logic and API routes.
- **Database:** MongoDB for a flexible, document-based storage system.
- **Styling:** Tailwind CSS for rapid and modern UI development.
- **Tools:** Postman for API testing and Git for version control.

2.5 Data Requirements

The system is designed to handle specific data points for every booking to ensure no information is lost:

1. **Customer Name:** To identify the owner.
2. **Contact Number:** For any emergency communication.
3. **Vehicle Number:** Used as the primary key for tracking.
4. **Service Type:** To help mechanics prepare the necessary tools/parts.
5. **Scheduled Date/Time:** To prevent workshop congestion.

3. System Design & Architecture

3.1 Introduction

System design is the bridge between the requirements and the actual product. In this phase, I focused on designing a robust architecture that ensures speed, security, and scalability. This chapter covers the high-level architecture of the Vehicle Service Booking System, the logical flow of data, and the structural design of the database.

3.2 High-Level System Architecture

The application is built on the **Three-Tier Architecture** model using the **MERN Stack**. This separation of concerns ensures that the user interface, the server logic, and the data storage operate independently yet interact seamlessly.

1. **Presentation Tier (Frontend):** Developed using **React.js**, this is where the user interacts with the system. It handles the UI components, state management, and client-side routing.
2. **Logic Tier (Backend):** Built with **Node.js** and **Express.js**, this acts as the brain of the system. It processes requests from the frontend, enforces security rules (like JWT and sanitization), and communicates with the database.
3. **Data Tier (Database): MongoDB Atlas** is used as the persistent storage. Being a NoSQL database, it stores booking records as JSON-like documents, which is ideal for the flexible nature of service records.

3.3 System Workflow (Data Flow)

To ensure the system is intuitive, I designed a logical flow for both customers and administrators:

- **Customer Journey:** A customer fills out the booking form $\rightarrow$ React sends a POST request to the Express API $\rightarrow$ Data is validated and saved in MongoDB $\rightarrow$ A success message is returned to the user.
- **Tracking Process:** Customer enters vehicle number $\rightarrow$ React sends a GET request $\rightarrow$ The backend queries MongoDB for that specific vehicle $\rightarrow$ The current status is fetched and displayed on a dynamic progress bar.
- **Administrative Flow:** Admin logs in $\rightarrow$ System verifies credentials and issues a **JWT Token** $\rightarrow$ Admin accesses the dashboard and views all bookings $\rightarrow$ Admin updates a status $\rightarrow$ A PUT request updates the record in MongoDB.

3.4 Database Design (Data Modeling)

Instead of traditional tables, I utilized **Mongoose** to define schemas for our MongoDB collections. This ensures that every document saved follows a specific structure.

3.4.1 The 'Bookings' Collection

This is the core collection of the system. Each booking document contains the following fields:

Field Name	Data Type	Purpose
customerName	String	Stores the owner's name
phone	String	Contact details for communication
vehicleNumber	String	Unique key used for tracking (Indexed)
serviceType	String	Category of service requested
date	Date	The scheduled date for the appointment
status	String	Defaulted to 'Pending'; updated by Admin

3.5 Security by Design

Architecture-level security was a top priority for me during the design phase:

- **Environment Segregation:** Sensitive information like the MongoDB URI and JWT Secret are never hardcoded; they are managed through a .env file.
- **Middleware Integration:** I designed the backend to pass every request through security middleware (Helmet and CORS) before it reaches the main logic, ensuring the system is protected from common web threats.

3.6 API Design Pattern

The backend is designed as a **RESTful API**. This means it uses standard HTTP methods (GET, POST, PUT, DELETE) to perform operations. This makes the system clean, organized, and easy to maintain or expand in the future.

4. Database Design

4.1 Introduction

Data is the heart of any management system. For this project, I chose **MongoDB**—a NoSQL, document-oriented database—because of its flexibility and ability to handle JSON-like data structures, which perfectly matches the JavaScript environment of the MERN stack. Unlike traditional relational databases that use tables and rows, MongoDB stores data in "Collections" and "Documents." This chapter details the data dictionary and the logical schema of the system.

4.2 Database Choice Justification

I opted for **MongoDB Atlas** (Cloud version) for several practical reasons I encountered during development:

- **Schema Flexibility:** As a developer, I could easily add new fields (like 'time' or 'notes') without having to re-design the entire table structure.
- **JSON-Friendly:** Since the frontend (React) and backend (Node) communicate using JSON, storing data in the same format eliminates the need for complex data conversion.
- **Scalability:** It allows the service center to grow from handling 10 bookings a day to thousands without performance lag.

4.3 Data Dictionary (Bookings Collection)

The primary collection in our database is Bookings. Below is the detailed data dictionary representing the structure of each booking record:

Field Name	Data Type	Requirement	Description
_id	ObjectId	Mandatory	Automatically generated unique ID by MongoDB.
customerName	String	Mandatory	Full name of the vehicle owner.
phone	String	Mandatory	Contact number (used for verification).
vehicleNumber	String	Mandatory	The unique registration number (Indexed for fast search).
serviceType	String	Mandatory	The category of service (e.g., Oil Change, Full

Field Name	Data Type	Requirement	Description
			Service).
date	Date	Mandatory	The scheduled appointment date.
status	String	Optional	Current progress: 'Pending', 'Approved', or 'Completed'.
createdAt	Timestamp	Optional	Automatically records when the booking was made.

4.4 Admin Schema (Authentication)

For the administrative dashboard access, I designed a separate Users collection to store credentials securely.

Field Name	Data Type	Description
username	String	Unique username for admin login.
password	String	Salted and hashed password using **Bcrypt**.

4.5 Data Relationships and Indexing

Although MongoDB is NoSQL, I implemented logical relationships and indexing to optimize the system:

- **Indexing:** I placed a **Unique Index** on the vehicleNumber field. This ensures that the search operation in the "Track Status" module happens almost instantaneously, even as the database grows.
- **Data Integrity:** I used **Mongoose Schemas** in the backend to act as a gatekeeper, ensuring that no document is saved to the database unless it meets all the mandatory field requirements defined in Section 4.3.

- **4.6 Security of Stored Data**

Protecting customer information was a key concern. I implemented two main layers of security at the database level:

1. **Encryption at Rest:** Using MongoDB Atlas's built-in encryption.
2. **Access Control:** The database is configured to only allow connections from specific IP addresses, and the connection string is hidden within the server's environment variables.

5. Implementation & API Details

5.1 Introduction

The implementation phase is where the design blueprints are converted into a functional software product. For this project, I followed a modular approach, developing the backend services first to ensure a solid foundation before building the interactive frontend. This chapter provides an in-depth look at the backend structure, key programming logics, and the RESTful API design that powers the system.

5.2 Backend Implementation

The backend was developed using **Node.js** and the **Express.js** framework. The primary goal was to create a "Stateless" API that handles requests efficiently and securely.

5.2.1 Server Configuration and Middleware

To ensure the server is production-ready, I implemented several critical middleware layers:

- **Helmet.js:** Used to set various HTTP headers for app security.
- **CORS:** Configured to allow cross-origin requests specifically from our frontend domain.
- **Express Rate Limit:** Implemented to prevent "Brute Force" attacks by limiting the number of requests a single IP can make.
- **Body Parser:** To handle and sanitize incoming JSON data payloads.

5.3 RESTful API Design

I designed the API following REST principles, ensuring each endpoint is intuitive and follows standard HTTP methods.

5.3.1 API Endpoints Table

The following table summarizes the core endpoints developed for the system:

Method	Endpoint	Description	Authentication
POST	/api/bookings	Submits a new service request.	Public
GET	/api/bookings/status/:id	Fetches status using vehicle number.	Public
POST	/api/auth/login	Authenticates admin and issues JWT.	Public
GET	/api/bookings	Retrieves all bookings for dashboard.	Admin (JWT)
PUT	/api/bookings/:id/status	Updates the service progress.	Admin (JWT)
DELETE	/api/bookings/:id	Removes a record from the database.	Admin (JWT)

5.4 Key Implementation Logic

5.4.1 Secure Database Connection

Instead of hardcoding the connection string, I used the dotenv library to pull the MONGO_URI from a secure environment file. This prevents sensitive credentials from being leaked if the code is shared or hosted on platforms like GitHub.

5.4.2 NoSQL Injection Protection

A custom middleware was developed to sanitize user inputs. This logic recursively checks the request body, params, and queries to remove any keys starting with $ or ., which are commonly used in NoSQL injection attacks to bypass authentication or delete data.

5.4.3 JWT-Based Authentication

For admin security, I implemented **JSON Web Tokens (JWT)**. When an admin logs in successfully, the server signs a token containing the user's ID. This token must be included in the

header of all protected requests (like updating a booking status), ensuring that only verified administrators can modify the data.

5.5 Frontend-Backend Integration

The communication between the React frontend and the Express backend is handled via the **Axios** library.

- **Asynchronous Calls:** Using async/await syntax to ensure the UI remains responsive while waiting for database responses.
- **Error Handling:** Implementing try-catch blocks to provide meaningful feedback to the user (e.g., displaying "Vehicle Number Not Found" instead of a generic server error).

6. User Interface (UI) Design

6.1 Introduction

The User Interface (UI) is the direct point of contact between the system and its users. For this project, the UI was designed using **React.js** and styled with **Tailwind CSS**. My primary focus was on "User-Centric Design," ensuring that the interface is clean, intuitive, and fully responsive across all devices (Mobile, Tablet, and Desktop). This chapter showcases the key screens of the application and explains their specific functionalities.

6.2 Design Philosophy

During the development of the UI, I followed three main principles:

- **Simplicity:** Minimalist layouts to avoid overwhelming the user with too much information.
- **Consistency:** Using a uniform color palette (Professional Blues and Greys) and typography across all pages.
- **Feedback:** Providing clear visual cues, such as loading spinners and toast notifications, to inform the user about the status of their actions.

6.3 Key Modules and Screens

6.3.1 Customer Booking Page

This is the entry point for customers. It features a clean form where users can input their service requirements.

- **Purpose:** To collect customer data and service details.
- **Features:** Date picker for scheduling, dropdown for service types, and real-time validation to prevent empty submissions.

6.3.2 Vehicle Tracking Page (The Status Tracker)

One of the unique features of this system is the real-time tracking page.

- **Functionality:** Users enter their unique Vehicle Number to query the database.
- **Visual Elements:** A dynamic **Progress Bar** was implemented to show the stage of the service.
    - **Pending:** Yellow indicator (Process initiated).
    - **Approved:** Blue indicator (Technician assigned).
    - **Completed:** Green indicator (Vehicle ready for pickup).

6.3.3 Admin Login Page

To ensure security, the administrative dashboard is protected by a dedicated login screen.

- **Authentication:** Integrated with the JWT-based backend.
- **Security:** Errors are handled gracefully (e.g., "Invalid Credentials") without revealing sensitive system information.

6.3.4 Admin Dashboard (The Control Center)

The dashboard provides a bird's-eye view of all operations.

- **Analytics Cards:** Displaying metrics like "Total Bookings," "Pending Approvals," and "Completed Services."
- **Management Table:** A comprehensive table listing all bookings with "Action" buttons to update status or delete records.
- **Real-time Updates:** When an admin changes a status, the change is reflected instantly on the customer-facing tracking page.

6.4 Responsive Design Implementation

Since many customers book services via their mobile phones, I used **Tailwind's Responsive Utilities** to ensure the layout shifts smoothly from a multi-column desktop view to a single-column mobile view. This ensures that buttons are "thumb-friendly" and text remains legible on smaller screens.

6.5 User Experience (UX) Enhancements

To make the system more interactive, I included:

- **Toast Notifications:** Using react-toastify to show success messages after a successful booking.
- **Empty States:** If no booking is found for a vehicle number, a clear "No Records Found" message with an icon is displayed instead of a blank screen.

7. Security, Testing, and Deployment

7.1 Introduction

Building a functional application is only half the battle; ensuring it is secure, bug-free, and accessible to users is the other half. This chapter covers the rigorous testing methods I followed, the security protocols implemented to protect user data, and the final deployment process to cloud platforms.

7.2 Security Implementation

Security was integrated into the system from the architecture level. I followed several industry-standard practices to ensure the application is resilient against common web vulnerabilities.

- **Authentication & Authorization:** Using **JSON Web Tokens (JWT)**, I ensured that administrative routes are only accessible to verified users. This prevents unauthorized access to customer data.
- **Data Sanitization:** To prevent **NoSQL Injection**, I implemented middleware that strips dangerous characters from incoming requests.
- **Environment Variable Management:** Sensitive keys such as the MongoDB connection string and JWT secret are stored in a .env file, ensuring they are never exposed in the source code.
- **Secure Headers:** By integrating **Helmet.js**, the system automatically sets secure HTTP headers to protect against Cross-Site Scripting (XSS) and clickjacking.

7.3 Testing and Quality Assurance

To ensure a smooth user experience, I conducted testing in two main phases:

7.3.1 API Testing (Back-End)

I used **Postman** to test every individual endpoint. I verified:

- **Success Scenarios:** Ensuring a 200 OK or 201 Created response when valid data is sent.
- **Error Handling:** Ensuring the server returns appropriate error messages (e.g., 401 Unauthorized) when invalid data or no token is provided.

7.3.2 System Integration Testing (Front-End to Back-End)

I performed "End-to-End" testing by simulating a real user's journey:

1. Submitting a booking form and checking if the data appears in the database.
2. Logging in as an admin and updating the status.
3. Refreshing the tracking page to see if the progress bar updated correctly.

7.4 Deployment

For the final project delivery, I deployed the application on cloud platforms to make it accessible via a public URL.

- **Back-End Deployment (Render):** The Node.js server was hosted on Render. This platform allows for seamless integration with GitHub, automatically re-deploying the server whenever a change is pushed.
- **Front-End Deployment (Vercel):** The React application was deployed on Vercel. Vercel provides a global CDN, ensuring the website loads quickly for users regardless of their location.
- **Database (MongoDB Atlas):** The data resides in a managed cloud cluster, providing 99.9% uptime and automated backups.

8. Conclusion and Future Enhancements

8.1 Conclusion

The development of the **Vehicle Service Booking System** has been a significant milestone in my journey as a full-stack developer. Over the course of this one-month internship, I successfully transformed a manual, error-prone process into a streamlined, secure, and user-friendly digital platform.

By utilizing the **MERN stack**, I was able to meet all the primary objectives set out at the beginning of the project. The system now allows customers to schedule appointments 24/7 and provides them with much-needed transparency through the real-time status tracking module. For the service center management, the administrative dashboard has replaced messy logbooks with an organized, data-driven interface.

Beyond the technical implementation, this project taught me the importance of **system security** (through JWT and sanitization) and **user experience** (through responsive design and visual progress tracking). It stands as a robust foundation for any vehicle service provider looking to modernize their operations.

8.2 Future Enhancements

While the current version of the system (V1.0) is stable and fully functional, software development is an iterative process. Based on user feedback and industry trends, I have identified several key areas for future growth:

8.2.1 Automated Communication Gateway

Currently, customers must manually check their status. In the next phase, I plan to integrate **Twilio (for SMS)** or **Nodemailer (for Email)**. This will allow the system to automatically alert the customer the moment an admin approves their booking or completes the service.

8.2.2 Online Payment Integration

To make the system a truly "end-to-end" solution, I aim to integrate a payment gateway like **Stripe** or **PayHere**. This would allow customers to settle their bills online before coming to pick up their vehicles, further reducing congestion at the service center.

8.2.3 Comprehensive Service History

Future versions will include a "Customer Account" feature. By creating a profile, users will be able to see a detailed history of every service their vehicle has undergone, including past dates, costs, and mechanic notes, which adds immense value to the vehicle's resale documentation.

8.2.4 Spare Parts & Inventory Management

For the admin side, adding an inventory module would be a massive upgrade. When a service is marked as "Completed," the system could automatically deduct the used parts (like oil filters or brake pads) from the digital inventory and alert the manager if stocks are low.

8.3 Final Thoughts

This project was more than just a requirement for my internship; it was an exercise in problem-solving and modern engineering. Seeing the system go from a simple database schema to a live, deployed application has been incredibly rewarding. It has solidified my understanding of the MERN stack and prepared me for more complex challenges in the field of software engineering.